Using Code and SQL Analysis Tools

Here you'll see how the rails-pg-extras[25] Ruby gem and command-line tool can be used to help with schema design and query design. The gem has been added to Rideshare.

In your terminal, run bin/rails pg_extras:cache_hit to look at index hit rate and table hit rate ratios. You want to see high hit ratios for both types.

A low index hit ratio could indicate there are too few indexes. PostgreSQL keeps a cache of indexes in memory and a low hit ratio means there are many cache misses. Refer to the gem documentation for more information.

Next run the bin/rails pg_extras:diagnose command to get an overall report for Rideshare. Good items are printed in green and improvement areas are printed in red.

Indexes may be skipped for usage in execution plans when they're suffering from very high estimated bloat. As you saw earlier, the solution in that case would be to rebuild the index.

Another analysis tool you saw earlier is pgbadger. PgBadger can be also be used for historical query analysis.

Queries are presented in P90, P95, and P99 percentile groupings. Focus on the P90 percentile first, before tackling optimization opportunities for less common P95 and P99 percentiles.

Wrapping Up

This chapter focused on helping you build your query observability tool chain. You automatically logged slow queries from the application. You found high impact slow queries using PGSS on the server. You learned how to manually and automatically collect samples and execution plans. You started to identify common performance problems like Missing Indexes and tactics to fix those problems.

You didn't quite yet dive into queries in depth, but you gained comfort modifying parameters and using the PostgreSQL log to identify information. You configured PGSS and auto_explain extensions to help you identify and fix problems.

You started to scratch the surface of query execution plans and how Indexing works.

25. https://github.com/pawurb/rails-pg-extras